tarefa_argumentos_parms

September 2, 2026

1. função para calcular o dobro

[11]:
```
def dobro(n):
    return 2*n

dobro(7)
```

[11]:
```
14
```

2. função para calcular a média

[10]:
```
def media(a, b, c):
    return (a+b+c)/3

media(3,2,7)
```

[10]:
```
4.0
```

3. função para verificar paridade

[9]:
```
def eh_par(n):
    if n % 2 == 0:
        return "é par"
    else:
        return "não é par"
eh_par(7)
```

[9]:
```
'não é par'
```

4. conversão de temperatura

[15]:
```
def celsius_para_fahrenheit(c):
    return (9/5)*c + 32

celsius_para_fahrenheit(30)
```

[15]:
```
86.0
```

5. área de um retângulo

[16]:
```
def area_retangulo(base, altura):
    return base*altura

area_retangulo(4, 5)
```

[16]: 20

6. parâmetro opcional

[21]:
```
def saudacao(nome, mensagem='olá,'):
    print(mensagem, nome)
saudacao('Rudnei')
```

```
olá, Rudnei
```

7. potência com expoente padrão

[24]:
```
def potencia(base, expoente=2):
    return base**expoente

potencia(4, 3)
```

[24]: 64

8. argumentos posicionais e nomeados

[28]:
```
def apresentar(nome, idade, cidade):
    print(f'Olá, meu nome é {nome}, tenho {idade} anos e moro em {cidade}.')

# a) usando apenas argumentos posicionais
apresentar('Leo', 23, 'Porto Alegre')

# b) usando apenas argumentos nomeados
apresentar(idade=79, cidade="São José dos Campos", nome="Laura")

# c) usando uma combinação de argumentos posicionais e nomeados
apresentar("Roberto", idade=27, cidade="Erechim")
```

```
Olá, meu nome é Leo, tenho 23 anos e moro em Porto Alegre.
Olá, meu nome é Laura, tenho 79 anos e moro em São José dos Campos.
Olá, meu nome é Roberto, tenho 27 anos e moro em Erechim.
```

9. valor padrão para desconto

[38]:
```
def preco_final(preco, desconto=0):
    return preco * (1 - desconto/100)

preco_final(50, 14)
```

[38]: 43.0

10. função com vários parâmetros opcionais

[ ]:
```
def produto(nome, preco, quantidade=1, desconto=0):
    return preco * quantidade * (1 - desconto/100)

produto("cachaça", 50, 5, 10)
```

[ ]:
```
225.0
```

11. modificação de um inteiro

[1]:
```
def modificar(x):
    x = x + 10

numero = 20
modificar(numero)
print(numero)
```

```
20
```

a variável `numero` não foi alterada porque recebeu um objeto do tipo `int`, que é imutável. ao fazer `x = x + 10`, um novo objeto é criado na memória para o qual a variável local `x` aponta. a variável de fora, `numero` não muda.

[32]:
```
def modificar_mutavel(x):
    return x + 10

numero = 20
numero = modificar_mutavel(numero)
print(numero)
```

```
30
```

agora, fazemos a função `modificar_mutavel` retornar um valor, que depois é atribuído à variável `numero`.

12. modificação de uma string

[13]:
```
def maiusculas(texto):
    texto = texto.upper()

hino = "glória do desporto nacional"
maiusculas(hino)
print(hino)
```

```
glória do desporto nacional
```

13. modificação de uma lista

[14]:
```
def adicionar(lista, valor):
    lista.append(valor)
```

```
numeros = [1, 2, 3]
adicionar(numeros, 4)
print(numeros)
```

```
[1, 2, 3, 4]
```

a lista original foi modificada porque `listé` um objeto mutável, ou seja, quando executamos `lista.append`, a própria lista `numeros` é modificada.

14. lista e atribuição

[15]:
```
def modificar(lista):
    lista = lista + [10]

numeros = [1,2,3]
modificar(numeros)
print(numeros)
```

```
[1, 2, 3]
```

a diferença aqui é que o comando `lista + [10]` cria um novo objeto para o qual a variável local (dentro da função) `lista` aponta. no exemplo anterior, o método `append` modifica o próprio objeto (a lista original).

15. lista com append

[16]:
```
def duplicar_elementos(lista):
    for i in range(len(lista)):
        lista[i] *= 2

lista = [1, 2, 3]
duplicar_elementos(lista)
print(lista)
```

```
[2, 4, 6]
```

16. modificação de um dicionário

[17]:
```
def atualizar_aluno(aluno, idade):
    aluno['idade'] = idade

aluno = {"nome": "Ana", "idade": 20}
atualizar_aluno(aluno, 21)
print(aluno)
```

```
{'nome': 'Ana', 'idade': 21}
```

17. dicionário vs atribuição

[22]:
```
def modificar(pessoa):
    pessoa = {"nome": "Carlos"}
```

```
dados = {"nome": "Ana"}
modificar(dados)
print(dados)
```

```
{'nome': 'Ana'}
```

o comando `{"nome": "Carlos"}` cria um novo objeto na memória, para o qual a variável local `pessoa` referencia. com isso, a variável de fora `dados` não é alterada.

18. função que modifica um conjunto

[25]:
```
def adicionar_elementos(conjunto, elementos):
    for elemento in elementos:
        conjunto.add(elemento)

numeros = {1, 2, 3}
adicionar_elementos(numeros, [3, 4, 5])
print(numeros)
```

```
{1, 2, 3, 4, 5}
```

19. função que retorna uma nova lista

[26]:
```
def pares(lista):
    return [x for x in lista if x % 2 == 0]

numeros = [1, 2, 3, 4, 5, 6]
resultado = pares(numeros)
print(resultado)
print(numeros)
```

```
[2, 4, 6]
[1, 2, 3, 4, 5, 6]
```

20. desafio: cadastro de alunos

[29]:
```
def cadastrar_aluno(alunos, nome, idade, curso="Computação"):
    alunos[nome] = {"idade": idade, "curso": curso}

alunos = {}
cadastrar_aluno(alunos, "Ana", 20)
cadastrar_aluno(alunos, "Carlos", 22, curso="Matemática")
print(alunos)
```

```
{'Ana': {'idade': 20, 'curso': 'Computação'}, 'Carlos': {'idade': 22, 'curso':
'Matemática'}}
```